

# Chapter 1: The Family of Vehicle Routing Problems

Vehicle Routing: Problems, Methods, and Applications  
2nd Edition, SIAM, 2014

Stefan Irnich   Paolo Toth   Daniele Vigo

Slides compiled for self-study

2014 (slides prepared 2026)

# Table of Contents I

- 1 1.1 Introduction
- 2 1.2 The Capacitated Vehicle Routing Problem (CVRP)
- 3 1.3 The Family of VRP
- 4 1.4 Objectives
- 5 1.5 Relationship to TSP and Bin Packing
- 6 1.6 Benchmark Instances
- 7 1.7 Python: Implementing CVRP Tools
- 8 1.8 Key Takeaways and Chapter Summary
- 9 Bibliography (Selected)

# What Is the Vehicle Routing Problem? I

The **Vehicle Routing Problem (VRP)** is one of the most studied problems in combinatorial optimisation, with widespread real-world applications in logistics, distribution, and transportation. A precise verbal definition is the following:

## Informal Definition of the VRP (Irnich, Toth, Vigo 2014)

**Given:** A set of *transportation requests* and a *fleet of vehicles*.

**Task:** Determine a set of *vehicle routes* to *perform* all (or some) transportation requests with the given vehicle fleet *at minimum cost*. In particular, decide *which vehicle handles which requests in which sequence* so that all vehicle routes can be *feasibly* executed.

In this type of problem the transportation requests to be served are generally concentrated in specific *points* of a road network. This contrasts with the *Arc Routing Problems (ARP)*, where requests are dispersed along the arcs, i.e., street segments of the underlying road network.

The four key components of any VRP instance are:

- 1 **Transportation requests** — what needs to be moved, and where.

# What Is the Vehicle Routing Problem? II

- ② **Fleet of vehicles** — how many vehicles, of what type, from what depot.
- ③ **Costs** — travel distance, time, fuel, driver wages, etc.
- ④ **Feasibility of routes** — physical constraints (capacity, hours, time windows).

# What Is the Vehicle Routing Problem? III

## Economic Relevance

The large number of real-world applications have widely shown that the use of computerised solution methods for the VRP, both at the planning and the operational levels, yields *substantial savings* in global transportation costs. The success of optimisation techniques is due not only to the power of modern computer systems and the full integration of information systems into operations, but also to the development of rigorous mathematical models that can take into account almost all characteristics of real-world distribution processes.

The chapter also serves as a foundation for the research on specific VRP variants described in later chapters. It presents basic models, identifies the main components that distinguish variants from each other, and provides a taxonomy of the VRP family.

## 1.2.1 Problem Statement I

The **Capacitated Vehicle Routing Problem (CVRP)** is the most fundamental and most-studied member of the VRP family. Every other variant can be understood as the CVRP with additional constraints or relaxations.

### Problem Statement (CVRP)

A set of  $n$  customers, each at a known location and each with a known *demand*  $d_i > 0$  (e.g., weight of goods to be delivered), must be served by a homogeneous fleet of  $K$  vehicles. Each vehicle has the same capacity  $Q$  (maximum load it can carry). All vehicles start and end at a single *depot*. The objective is to find routes for all vehicles such that:

- every customer is visited exactly once,
- the total load on any single route does not exceed  $Q$ ,
- the total travel cost (sum of distances) is minimised.

**Graph model.** The CVRP is defined on a complete directed graph  $G = (V, A)$ , where  $V = \{0, 1, \dots, n\}$  is the set of nodes and  $A = \{(i, j) : i, j \in V, i \neq j\}$  is the set of arcs (directed edges). Here:

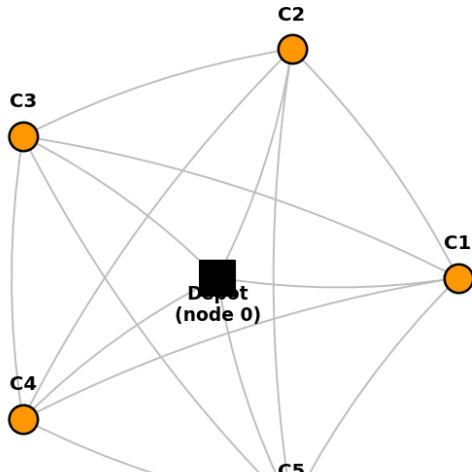
- Node 0 is the *depot* — the starting and ending point for every vehicle.

## 1.2.1 Problem Statement II

- Nodes  $1, 2, \dots, n$  are the *customers*.
- Each arc  $(i, j)$  has an associated *travel cost*  $c_{ij} \geq 0$ , which in the most common case is the Euclidean distance between  $i$  and  $j$ .

## 1.2.1 Problem Statement III

**Complete Directed Graph  $G=(V, A)$  for the CVRP**  
 **$V = \{0, 1, \dots, n\}$ , depot = node 0, customers = nodes 1..n**





## 1.2.2 Mathematical Formulation (VRP-Flow Models) I

The CVRP can be formulated as an integer linear programme. We present two well-known formulations.

### Sets and parameters:

- $V = \{0, 1, \dots, n\}$ : all nodes ( $0 = \text{depot}$ ,  $1 \dots n = \text{customers}$ ).
- $V' = \{1, \dots, n\}$ : customer nodes only (the prime notation means “without the depot”).
- $A$ : set of all directed arcs  $(i, j)$  with  $i \neq j$ .
- $c_{ij}$ : travel cost of arc  $(i, j)$ . In Euclidean instances,  $c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ .
- $d_i$ : demand of customer  $i$  (in units compatible with  $Q$ ; for the depot,  $d_0 = 0$ ).
- $Q$ : vehicle capacity (maximum load any single vehicle may carry).
- $K$ : number of available vehicles (or, in a free-fleet formulation, a sufficiently large upper bound).

### Decision variables:

- $x_{ij} \in \{0, 1\}$ : equals 1 if some vehicle travels directly from node  $i$  to node  $j$ , and 0 otherwise. The subscript “ $ij$ ” (pronounced “x sub  $i$  j”) identifies the directed arc from  $i$  to  $j$ .
- $u_i \geq 0$ : an auxiliary integer variable for the Miller–Tucker–Zemlin (MTZ) subtour-elimination formulation (explained below).

## 1.2.2 Mathematical Formulation (VRP-Flow Models) II

### VRP-Flow Formulation (MTZ-style)

$$\text{(VRP)} \quad \min \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} x_{ij} \quad (1.1)$$

$$\sum_{j \in V, j \neq i} x_{ij} = 1, \quad \forall i \in V' \quad (1.2)$$

$$\sum_{i \in V, i \neq j} x_{ij} = 1, \quad \forall j \in V' \quad (1.3)$$

$$\sum_{j \in V'} x_{0j} = K \quad (1.4)$$

$$\sum_{i \in V'} x_{i0} = K \quad (1.5)$$

$$u_i - u_j + Q x_{ij} \leq Q - d_j, \quad \forall i \in V', j \in V', i \neq j \quad (1.6)$$

$$d_i < u_i < Q, \quad \forall i \in V' \quad (1.7)$$

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) I

A more compact formulation uses only the binary variables  $x_{ij}$ . It requires a stronger subtour elimination mechanism, typically based on *capacity inequalities* (Laporte and Nobert 1987; Toth and Vigo 2002):

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) II

### Two-Index Vehicle-Flow Formulation

$$\text{(VRP)} \quad \min \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (1.1')$$

$$\sum_{j \in V} x_{ij} = 1, \quad \forall i \in V' \quad (1.2')$$

$$\sum_{i \in V} x_{ij} = 1, \quad \forall j \in V' \quad (1.3')$$

$$\sum_{i \in S, j \notin S} x_{ij} \geq r(S), \quad \forall S \subseteq V', S \neq \emptyset \quad (\text{Capacity cut})$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in A \quad (1.8')$$

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) III

The **capacity cut constraint** says: for any non-empty subset  $S$  of customers, the number of arcs leaving  $S$  towards nodes outside  $S$  must be at least  $r(S)$ . Here  $r(S)$  is the *minimum number of vehicles needed to serve all customers in  $S$* , computed as:

$$r(S) = \left\lceil \frac{\sum_{i \in S} d_i}{Q} \right\rceil$$

where  $\lceil \cdot \rceil$  (ceiling function) rounds up to the nearest integer.

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) IV

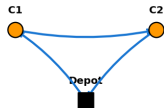
### Why do we need subtour elimination?

Without subtour elimination, the constraints (1.2') and (1.3') alone only guarantee that each customer is entered and exited exactly once — but they do *not* prevent solutions where customers form isolated loops disconnected from the depot. For example, customers  $\{3, 4\}$  might form a self-contained cycle  $3 \rightarrow 4 \rightarrow 3$  that never touches the depot. Such a “solution” is infeasible for routing but satisfies the in/out-degree constraints.

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) V

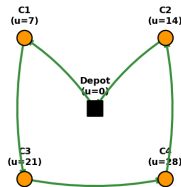
### MTZ Constraints: Eliminating Subtours in VRP/TSP Formulations

Without MTZ: subtours are allowed



INVALID: subtour  $\{C_3, C_4\}$  disconnected from depot

With MTZ: single connected route



Route: 0 -> 1 -> 3 -> 4 -> 2 -> 0

The left panel shows an invalid solution containing a subtour  $\{C_3, C_4\}$  disconnected from the depot. The right panel shows a valid single-route solution enforced by the MTZ auxiliary variables  $u_i$ .

## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) VI

### Worked Example: MTZ Variables

Consider 4 customers with demands  $d_1 = 3$ ,  $d_2 = 4$ ,  $d_3 = 3$ ,  $d_4 = 4$  and capacity  $Q = 10$ . A feasible route is  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ , with cumulative loads:

$$u_1 = 3, \quad u_2 = 7, \quad u_3 = 10, \quad u_4 = 14.$$

But  $u_4 = 14 > Q = 10$ , so this route *violates* the capacity constraint. The MTZ constraint (1.6) catches this: for arc  $(3, 4)$ , we need  $u_3 - u_4 + Q \cdot x_{34} \leq Q - d_4$ , i.e.,  $10 - 14 + 10 \cdot 1 = 6 \leq 10 - 4 = 6$ . Equality holds, so the constraint is satisfied — but only because  $u_3$  and  $u_4$  encode the actual load. If we tried to pretend  $u_3 = 3$  and  $u_4 = 3$  in a subtour, the constraint would be violated, correctly flagging the subtour as infeasible.



## 1.2.2 Compact Formulation (Two-Index Vehicle-Flow) VII

### Complexity Note

The CVRP is NP-hard (it generalises the Travelling Salesman Problem, which is itself NP-hard). The capacity-cut formulation has an *exponential* number of subtour elimination constraints (one for each non-empty subset  $S \subseteq V'$ ), but in practice they are added *lazily* via a branch-and-cut algorithm — only violated constraints are generated.

## 1.2.3 Extensive Formulation (Set Partitioning) I

An alternative, elegant formulation — the **Extensive Formulation** or *Set Partitioning* model — was first proposed by Balinski and Quandt (1964) and later used by Gilmore and Gomory (1965) as a starting point for column-generation approaches.

The key idea is to *enumerate all feasible routes* and then select the best combination that covers every customer exactly once.

### Notation:

- Let  $\Omega$  (omega, pronounced “oh-may-gah”) be the set of all feasible routes (each route is a sequence of customers starting and ending at the depot, with total load  $\leq Q$ ).
- For each route  $r \in \Omega$ , let  $c_r$  be its total travel cost.
- For each route  $r \in \Omega$  and customer  $i$ , let  $a_{ir} \in \{0, 1\}$  be an indicator:  $a_{ir} = 1$  if customer  $i$  is visited on route  $r$ , and 0 otherwise.
- Decision variable:  $\lambda_r$  (lambda, pronounced “lam-dah”)  $\in \{0, 1\}$  equals 1 if route  $r$  is selected.

## 1.2.3 Extensive Formulation (Set Partitioning) II

### Set-Partitioning Formulation (SP)

$$(SP) \quad \min \sum_{r \in \Omega} c_r \lambda_r \quad (1.9)$$

$$\sum_{r \in \Omega} a_{ir} \lambda_r = 1, \quad \forall i \in V' \quad (1.10)$$

$$\sum_{r \in \Omega} \lambda_r \leq K \quad (1.11)$$

$$\lambda_r \in \{0, 1\}, \quad \forall r \in \Omega \quad (1.12)$$

## 1.2.3 Extensive Formulation (Set Partitioning) III

### Explanation:

**Objective (1.9).** Minimise the total cost of all selected routes.

**Constraint (1.10): coverage.** For each customer  $i$ , the sum of  $a_{ir}\lambda_r$  over all routes  $r$  must equal exactly 1. In plain words: customer  $i$  must appear in exactly one selected route. (This is the “partition” in set partitioning — customers are partitioned among routes.)

**Constraint (1.11): fleet size.** At most  $K$  routes are selected (at most  $K$  vehicles depart).

### Why This Formulation?

The SP formulation is very clean and its LP relaxation provides extremely tight lower bounds. Modern exact VRP solvers are almost always based on column generation applied to the LP relaxation of the SP formulation, combined with branch-and-bound. The column generation subproblem — finding the route  $r$  with the most negative reduced cost — is a constrained shortest-path problem, itself NP-hard but solvable with dynamic programming via labelling algorithms.

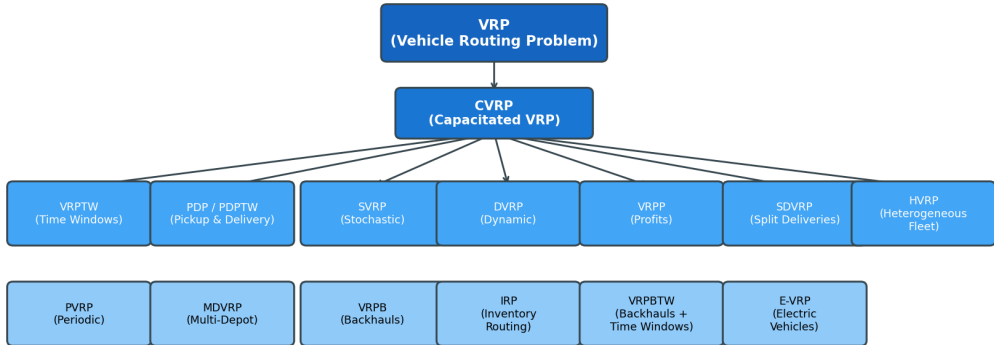
**Practical limitation:** The set  $\Omega$  of all feasible routes can be astronomically large — exponential in  $n$ . Therefore the SP model cannot be solved directly by enumerating all columns. Instead, *column generation* dynamically adds only the most promising routes during LP solving.

# Overview of the VRP Family I

Over decades of research, the basic CVRP has been extended in many directions to capture additional real-world constraints and objectives. The resulting collection of variants is referred to as **the family of vehicle routing problems**.

# Overview of the VRP Family II

## The Family of Vehicle Routing Problems (VRP)



*Each variant adds one or more real-world constraints to the basic CVRP.  
Many logistics and distribution problems reduce to one of these variants.*

## 1.3.1 Network and Customer Characteristics I

Before discussing specific variants, it is useful to understand the *building blocks* that differentiate VRP variants. The first building block concerns the characteristics of the underlying network and the customers.

**Transportation requests.** In the basic CVRP, each customer  $i$  has a single demand  $d_i$  that must be *delivered* from the depot. More general settings include:

- **Delivery and Collection:** some customers receive goods (delivery, also called *linehaul*) while others send goods back to the depot (collection, also called *backhaul*). The *VRP with Backhauls (VRPB)* requires all deliveries on a route to be completed before any pickups begin.
- **Pickup and Delivery:** goods are transported between pairs of locations  $(p_i, d_i)$  — picked up at  $p_i$  and delivered to  $d_i$ . The vehicle must visit  $p_i$  before  $d_i$  on the same route. This is the *Pickup and Delivery Problem (PDP)*.
- **Non-split Deliveries:** each customer's demand must be served by a single vehicle in a single visit. This is the standard CVRP assumption.
- **Split Deliveries:** a customer's demand may be split across multiple vehicles. This can reduce total distance but requires coordination.

## 1.3.1 Network and Customer Characteristics II

**Loading constraints.** The vehicle is not just a container with a scalar capacity. Real vehicles have a 3-dimensional interior, and the order in which goods are loaded matters:

- *Last-In-First-Out (LIFO)*: if goods are stacked, the last item loaded must be the first unloaded. This imposes a constraint on the order of deliveries within a route.
- *2D/3D bin packing*: goods must physically fit in the vehicle's cargo space. This couples the routing problem with a bin packing problem.

**Multiple depots.** In the *Multi-Depot VRP (MDVRP)*, there are several depots. Vehicles can start and end at different depots, and customers must be assigned to depots.

**Alternative delivery locations.** A customer can be served at one of several alternative locations (e.g., collection points, lockers). This relaxes the geographical constraint and can reduce total distance.



## 1.3.2 Time Windows (VRPTW) I

The **Vehicle Routing Problem with Time Windows (VRPTW)** is probably the most widely studied VRP variant. It extends the CVRP by associating a *time window*  $[a_i, b_i]$  with each customer  $i$  and with the depot.

### Time Window Constraint

A vehicle must arrive at customer  $i$  no earlier than  $a_i$  (*earliest service time*) and no later than  $b_i$  (*latest service time*). If it arrives before  $a_i$ , it must *wait* until  $a_i$  to begin service. If it cannot arrive by  $b_i$ , the solution is *infeasible*.

### Additional notation for VRPTW:

- $s_i$ : service time at customer  $i$  (time spent unloading goods).
- $t_{ij}$ : travel time on arc  $(i, j)$  (may differ from cost  $c_{ij}$ ).
- $w_i \geq 0$ : waiting time at customer  $i$  (non-negative; vehicle must wait if it arrives early).
- $T_i$ : time at which service *begins* at customer  $i$ .

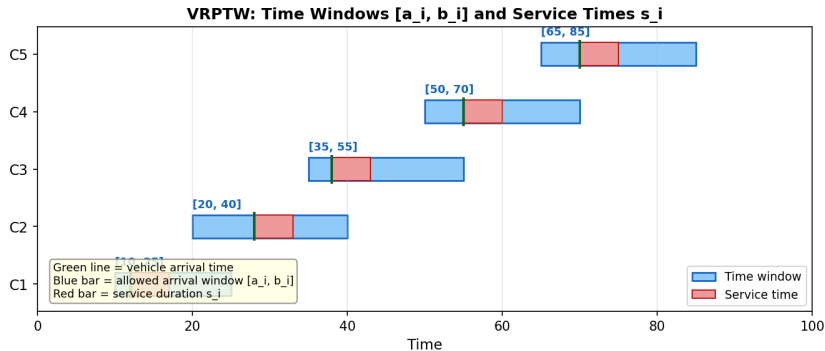
## 1.3.2 Time Windows (VRPTW) II

The service-start time propagates along a route:

$$T_j = \max(a_j, T_i + s_i + t_{ij})$$

This says: service at  $j$  begins either at the earliest allowed time  $a_j$ , or as soon as the vehicle arrives (after completing service at  $i$ , travelling to  $j$ ), whichever is later. The max function captures the *waiting* behaviour.

## 1.3.2 Time Windows (VRPTW) III



The figure shows time windows  $[a_i, b_i]$  (blue bars) for five customers. The green vertical line marks the vehicle's arrival time at each customer. The red bar shows service duration  $s_i$ .

**Types of time windows:**

## 1.3.2 Time Windows (VRPTW) IV

- *Hard time windows*: violation of  $b_i$  makes the solution infeasible. This is the standard VRPTW constraint.
- *Soft time windows*: arriving after  $b_i$  is allowed but incurs a penalty cost proportional to the delay. This is a common practical extension (e.g., penalise late deliveries by \$10 per minute).

### Key Insight: Why Time Windows Are Hard

Time windows create *cascading dependencies* among the customers on a route — a delay at one customer propagates and may push arrival at every subsequent customer outside its window. This makes the VRPTW significantly harder than the CVRP. Benchmark instances from Solomon (1987) remain standard for evaluating algorithms.

## 1.3.2 Time Windows (VRPTW) V

### VRPTW Formulation variables (extending CVRP):

- $x_{ij} \in \{0, 1\}$ : arc usage (same as CVRP).
- $T_i \geq 0$ : service start time at customer  $i$ .

The additional constraints for the VRPTW are:

$$\begin{aligned} T_i + s_i + t_{ij} - M(1 - x_{ij}) &\leq T_j, \quad \forall i \in V, j \in V', i \neq j \\ a_i &\leq T_i \leq b_i, \quad \forall i \in V \end{aligned}$$

where  $M$  (capital M) is a sufficiently large constant (a *big-M* coefficient). The first constraint says: if the vehicle uses arc  $(i, j)$  (i.e.,  $x_{ij} = 1$ ), then  $T_j \geq T_i + s_i + t_{ij}$  — time at  $j$  is at least time at  $i$  plus service time plus travel time. When  $x_{ij} = 0$ , the constraint is rendered inactive by the  $-M$  term.

### 1.3.3 Pickup and Delivery (PDP / PDPTW) I

In the basic CVRP, goods flow *from the depot to customers*. The **Pickup and Delivery Problem (PDP)** generalises this by allowing goods to flow between any pair of locations in the network.

#### Pickup and Delivery Problem (PDP)

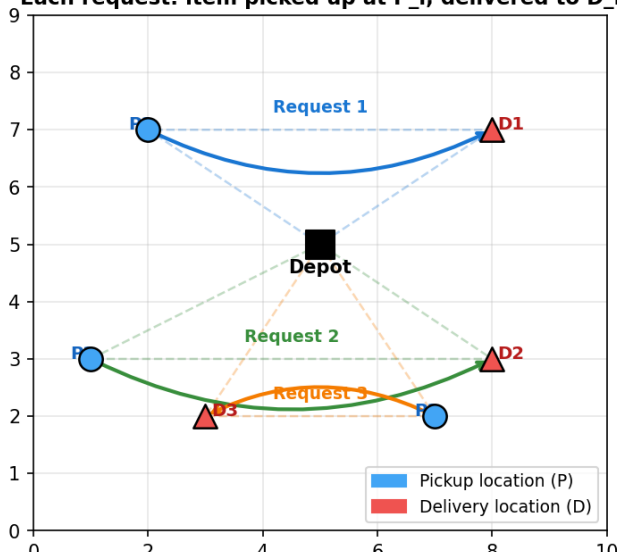
Each transportation request  $r = (p_r, d_r, q_r)$  consists of:

- $p_r$ : pickup location (the origin — where goods are collected).
- $d_r$ : delivery location (the destination — where goods are dropped off).
- $q_r$ : quantity of goods (demand for this request).

A vehicle that handles request  $r$  must visit  $p_r$  *before*  $d_r$  on the same route, and must carry the goods from  $p_r$  to  $d_r$  without exceeding capacity.

## 1.3.3 Pickup and Delivery (PDP / PDPTW) II

**Pickup and Delivery Problem (PDP)**  
Each request: item picked up at  $P_i$ , delivered to  $D_i$



## 1.3.4 Stochastic VRP (SVRP) I

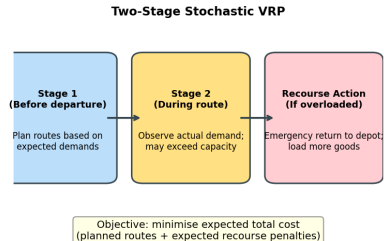
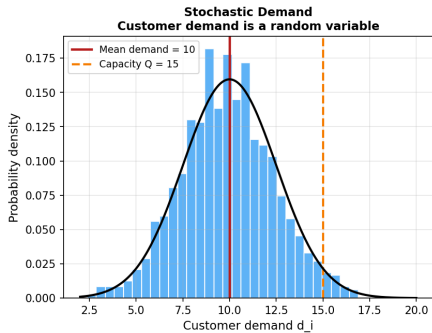
In practice, some input data — customer demands, travel times, or even which customers will request service — may not be known precisely in advance. The **Stochastic VRP (SVRP)** models this uncertainty explicitly.

### Three Main Sources of Stochasticity

- 1 **Stochastic demands:** Customer  $i$  has a random demand  $D_i$  (capital  $D$  to denote a random variable) drawn from a known probability distribution. The mean and variance of  $D_i$  are known at planning time, but the actual value is revealed only when the vehicle arrives at  $i$ .
- 2 **Stochastic travel times:** Travel time on arc  $(i, j)$  is a random variable, modelling traffic congestion or weather effects.
- 3 **Stochastic customers:** Not all customers in the potential set will actually request service. Customer  $i$  is present with probability  $p_i$  ( $p$  sub  $i$ ).



## 1.3.4 Stochastic VRP (SVRP) II



## 1.3.4 Stochastic VRP (SVRP) III

### Two-Stage Stochastic Programming Framework:

The SVRP is naturally modelled as a *two-stage stochastic programme*:

- **Stage 1 (here-and-now):** Before any uncertainty is revealed, plan routes based on expected/known data. These are the *first-stage* decisions.
- **Stage 2 (wait-and-see / recourse):** Once uncertainty is revealed during route execution, take *recourse actions* to restore feasibility.

The most common recourse action for the SVRP with stochastic demands is the *vehicle return to depot*: if a vehicle arrives at customer  $i$  and finds that its remaining capacity is insufficient to serve  $i$ 's actual demand, it returns to the depot to reload, then comes back to  $i$ .

### Objective Function of the SVRP

$$\min c(\text{first-stage route}) + \mathbb{E}_{\mathbf{D}}[Q(\text{route}, \mathbf{D})]$$

Here  $\mathbb{E}$  (blackboard-bold E) denotes the *expected value* (mathematical expectation) over the random vector  $\mathbf{D}$  (bold  $D$ ) of all customer demands.  $Q(\text{route}, \mathbf{D})$  is the recourse cost (extra distance due to emergency depot returns) given the realised demands  $\mathbf{D}$ .

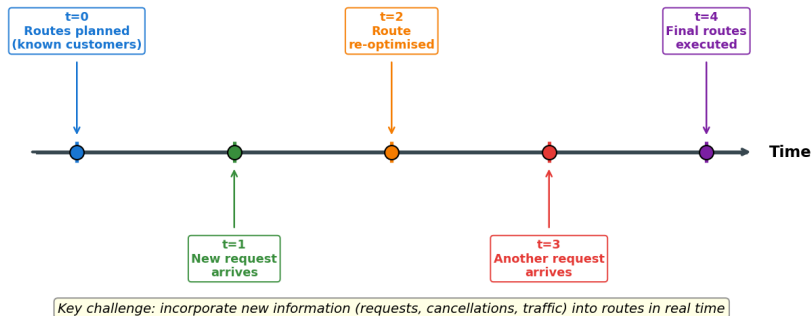
## 1.3.4 Stochastic VRP (SVRP) IV

The key challenge of the SVRP is that  $\mathbb{E}[Q]$  is generally hard to compute analytically and must be estimated by Monte Carlo simulation or approximated by closed-form bounds.

## 1.3.5 Dynamic VRP (DVRP) I

The **Dynamic VRP (DVRP)** arises when information about transportation requests becomes available *during the execution of routes* — not all requests are known at the time of planning.

**Dynamic VRP (DVRP): Requests Arrive During Execution**  
Routes must be continuously updated as new information arrives



## 1.3.5 Dynamic VRP (DVRP) II

**What makes a VRP dynamic?** The degree of dynamism (DoD) characterises how much of the routing information is known in advance. Formally, if  $n_d$  (n-sub-d) is the number of requests known only during execution, and  $n$  is the total number of requests, then:

$$\text{DoD} = \frac{n_d}{n}$$

A DoD of 0 means the problem is fully static (all information known upfront). A DoD of 1 means all requests arrive online with no advance information.

## 1.3.5 Dynamic VRP (DVRP) III

### Real-time management decisions in DVRP:

- 1 **Acceptance/rejection:** Decide in real time whether to accept a new request.
- 2 **Route modification:** Insert the new request into the best position in an existing route, or create a new route.
- 3 **Re-optimisation:** Periodically re-optimize all routes to incorporate accumulated new information.

### Relationship to stochastic VRP: Stochastic VRPs and dynamic VRPs are related but distinct:

- In the *stochastic* VRP, uncertainty is modelled *probabilistically* and taken into account at planning time.
- In the *dynamic* VRP, new information simply *arrives* during execution without a prior probabilistic model.
- A problem can be both stochastic *and* dynamic (e.g., the demand at a newly revealed customer is itself uncertain).

## 1.3.5 Dynamic VRP (DVRP) IV

### Practical Applications

Dynamic VRPs model taxi dispatching, ambulance routing, same-day delivery, ride-hailing (e.g., Uber/Lyft), and emergency vehicle routing. Real-time algorithms must respond within seconds to each new request.

## 1.3.6 VRP with Profits (VRPP) I

In the standard CVRP and VRPTW, *all* customers must be served. The **VRP with Profits (VRPP)** relaxes this constraint: each customer  $i$  has a *profit* (or prize)  $\pi_i$  ( $\pi_i$ , pronounced “pie”) associated with visiting it, and the planner decides which customers to visit.

### VRPP Objective

Maximise the total collected profit minus the total travel cost:

$$\max \sum_{i \in S} \pi_i - \sum_{(i,j) \in \text{routes}} c_{ij}$$

where  $S \subseteq V'$  is the set of *selected* customers (customers that are actually visited). The planner is free to skip any customer, but skipping customer  $i$  means forgoing profit  $\pi_i$ .

### Two main sub-variants:

- **Orienteering Problem (OP):** A single vehicle, a time limit  $T_{\max}$ , and profits  $\pi_i$  at nodes. The vehicle starts and ends at the depot. Maximise the total profit collected within the time budget. This is named after the sport of orienteering, where a competitor collects checkpoints within a time limit.



## 1.3.6 VRP with Profits (VRPP) II

- **Profitable Tour Problem (PTP):** Multiple vehicles; choose which customers to visit to maximise net profit (collected profits minus routing costs). No hard upper bound on number of vehicles.

### Key Insight

The VRPP trades off *visit benefit* (profit) against *visit cost* (distance). A customer far from the route is only worth visiting if its profit  $\pi_i$  exceeds the extra distance  $\delta c$  incurred by detouring to it. The formula for the detour cost of inserting customer  $i$  between customers  $j$  and  $k$  is:  $\delta c = c_{ji} + c_{ik} - c_{jk}$ .

## 1.3.7 Split Deliveries (SDVRP) I

In the standard CVRP, each customer's full demand must be served in a single visit by a single vehicle. The **Split Delivery VRP (SDVRP)** relaxes this assumption.

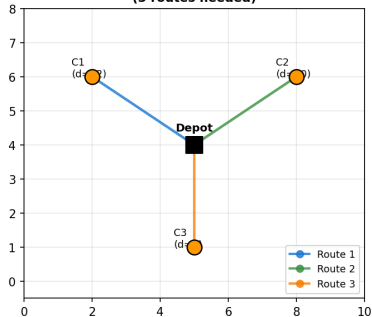
### Split Delivery

In the SDVRP, a customer  $i$  with demand  $d_i > Q$  (greater than vehicle capacity) *must* be served by multiple vehicles. More generally, even when  $d_i \leq Q$ , it may be *beneficial* to split a customer's demand across vehicles to reduce total routing distance.

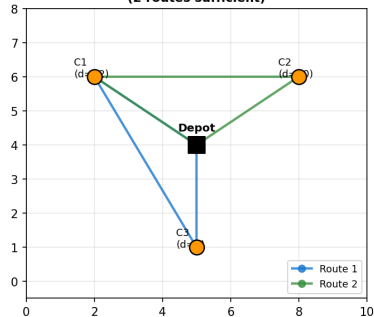
## 1.3.7 Split Deliveries (SDVRP) II

Split Deliveries: customer demand can be served by multiple vehicles ( $Q=15$ )

Without Split Deliveries  
(3 routes needed)



With Split Deliveries  
(2 routes sufficient)



## 1.3.7 Split Deliveries (SDVRP) III

### When does splitting help?

Consider 3 customers with demands  $d_1 = 12$ ,  $d_2 = 10$ ,  $d_3 = 8$  and vehicle capacity  $Q = 15$ . Without splitting, we need 3 vehicles (one per customer, since no pair fits in one vehicle:  $d_1 + d_2 = 22 > Q$ ,  $d_1 + d_3 = 20 > Q$ ,  $d_2 + d_3 = 18 > Q$ ).

With splitting:

- Route 1: serve customer 1 partially (7 units) and customer 3 fully (8 units). Load =  $7 + 8 = 15 = Q$ .
- Route 2: serve customer 2 fully (10 units) and remaining demand of customer 1 (5 units). Load =  $10 + 5 = 15 = Q$ .

Only 2 vehicles needed — a saving of one vehicle.

### Modelling Complexity

Split deliveries significantly increase solution space complexity because the quantity delivered to each customer becomes a continuous decision variable, on top of the combinatorial routing decisions. However, they can produce substantially cheaper solutions for large-demand customers.

## 1.3.8 Heterogeneous Fleet (HVRP) I

The standard CVRP assumes all vehicles are identical (same capacity  $Q$ , same cost per kilometre). In practice, fleets are often *heterogeneous*: different vehicle types have different capacities, fixed costs, and variable travel costs.

### Heterogeneous Fleet VRP (HVRP) Vehicles differ in capacity, fixed cost, and variable travel cost

Small Van  
Capacity  $Q=10$

Fixed cost: 50/routeVariablecost: 3/km

Medium Truck  
Capacity  $Q=20$

Fixed cost: 100/routeVariablecost: 2/km

Large Truck  
Capacity  $Q=40$

Fixed cost: 150/routeVariablecost: 1/km

Key challenge: assign customers to vehicle types to minimise total cost  
(larger capacity => higher fixed cost but fewer trips needed)

## 1.3.8 Heterogeneous Fleet (HVRP) II

### Heterogeneous Fleet VRP (HVRP)

There are  $T$  vehicle types. Vehicle of type  $t$  has:

- capacity  $Q_t$  (Q-sub-t): maximum load.
- fixed cost  $f_t$  (f-sub-t): cost incurred whenever a vehicle of type  $t$  is used, regardless of its route.
- variable cost  $v_t$  (v-sub-t): cost per unit of distance travelled by a type- $t$  vehicle.

## 1.3.8 Heterogeneous Fleet (HVRP) III

### Two sub-variants:

- **Fleet Size and Mix VRP (FSM):** The number of available vehicles of each type is *unlimited*. The optimiser decides how many of each type to use. This is called a “fleet composition” problem.
- **Heterogeneous VRP (HVRP):** The number of vehicles of each type is *fixed* in advance. The optimiser only decides which type serves which customers.

### Objective function for HVRP:

$$\min \sum_{t=1}^T f_t k_t + \sum_{r \in \Omega} c_r \lambda_r$$

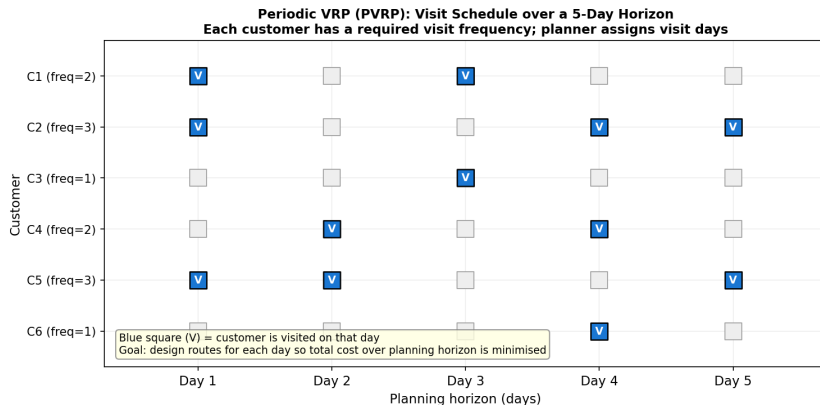
where  $k_t$  is the number of vehicles of type  $t$  used, and  $c_r$  is the cost of route  $r$  (using the appropriate variable cost for the assigned vehicle type).

### Trade-off: Capacity vs. Cost

Using a large truck reduces the number of routes needed (fewer vehicles), but incurs a higher fixed cost. Using many small vans avoids fixed costs but requires more routes and therefore more total travel. The HVRP must balance these competing incentives.

## 1.3.9 Periodic VRP (PVRP) I

The **Periodic VRP (PVRP)** extends the CVRP to a multi-day planning horizon. Each customer must be visited a specified number of times (*frequency*) over the planning period, and the planner must decide *on which days* to visit each customer.





## 1.3.9 Periodic VRP (PVRP) II

**Example:** A customer who needs deliveries 3 times per week (frequency = 3) in a 5-day horizon can be visited on days  $\{1, 2, 3\}$ ,  $\{1, 2, 5\}$ ,  $\{1, 3, 5\}$ ,  $\{2, 3, 4\}$ , etc. These are the *feasible visit combinations* for this customer.

## 1.3.9 Periodic VRP (PVRP) III

### PVRP Formulation

Let  $\mathcal{T} = \{1, 2, \dots, T\}$  be the set of days in the planning horizon. For each customer  $i$ , let  $\mathcal{P}_i$  be the set of feasible visit combinations (day-subsets of  $\mathcal{T}$  with the right frequency). A *schedule* for customer  $i$  selects one combination  $p \in \mathcal{P}_i$ .

The PVRP must simultaneously:

- 1 Assign each customer a visit combination (which days to visit).
- 2 Design one route per day that covers all customers assigned to that day.

The objective is to minimise the total routing cost over all days.

### Why PVRP Is Harder Than CVRP

The PVRP has an additional combinatorial dimension: the assignment of customers to days. This creates interdependencies across days — choosing a poor visit schedule for one customer can inflate routing costs on all affected days. The PVRP is a two-level combinatorial optimisation problem.

## 1.3.10 Intra-route Constraints I

So far we have focused on *inter-route* constraints (which routes to create, how many vehicles, capacity). But many practical problems impose additional *intra-route* constraints that restrict the *ordering* of customers within a single route.

**Loading constraints — Last-In-First-Out (LIFO).** When loading a van, goods placed at the back of the vehicle are accessed last. If the vehicle visits customers in the order  $i_1, i_2, \dots, i_k$ , then goods for customer  $i_k$  must be loaded *last* (closest to the door). Formally, the LIFO constraint requires that goods are loaded in reverse order of delivery. This rules out many otherwise optimal routes and is known to increase routing costs significantly.

**First-In-First-Out (FIFO).** In some contexts (e.g., fragile stacked goods), the loading sequence must follow the delivery sequence. FIFO loading constrains routes differently from LIFO.

**Precedence constraints.** As in the PDP, customer  $j$  may only be visited *after* customer  $i$  on the same route. This can model assembly-line-style logistics or sequential operations.

**Synchronisation constraints.** Two vehicles may need to be at the same location at the same time (e.g., for a vehicle transfer, or for a two-person job). Synchronisation constraints link the routes of different vehicles.

## 1.4 Objectives in VRP I

The objective function is a fundamental component of any VRP formulation. The choice of objective reflects the economic goal of the routing planner.

### Single Objective: Minimise Total Distance

The most common objective in the research literature:

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

This minimises the total distance (or equivalently, total travel cost) driven by all vehicles. When  $c_{ij}$  represents fuel cost, this objective directly minimises fuel expenditure.

## 1.4 Objectives in VRP II

### Minimise Number of Vehicles

A secondary, hierarchically superior objective used in many benchmark studies:

$$\min K \quad (\text{number of vehicles used})$$

In a lexicographic (two-level) objective, first minimise  $K$ , then minimise total distance. This reflects the reality that reducing fleet size frees expensive capital assets.

## 1.4 Objectives in VRP III

### Other common objectives:

- **Minimise total time (makespan):**  $\min \max_r T_r$ , where  $T_r$  is the completion time of route  $r$ . This minimises the time until the last vehicle returns. Relevant for emergency services.
- **Minimise total waiting time:**  $\min \sum_i w_i$ , where  $w_i$  is the waiting time at customer  $i$ . Important in time-window scenarios.
- **Maximise profit (VRPP):**  $\max \sum_{i \in S} \pi_i - \sum c_{ij} x_{ij}$  as discussed in Section 1.3.6.
- **Minimise emissions:** an increasingly important objective modelling CO<sub>2</sub> or other pollutants as a function of speed, load, and distance.
- **Balanced load:** minimise the imbalance among vehicle workloads. Relevant for driver satisfaction and labour regulations.

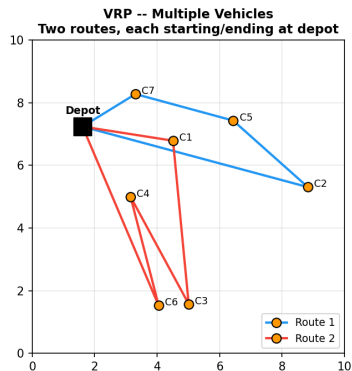
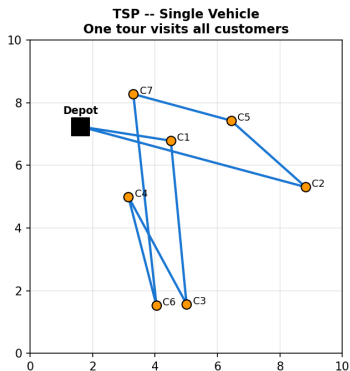
### Multi-Objective VRP

Real-world VRPs almost always involve multiple, conflicting objectives — for example, minimising cost while also minimising makespan and maximising customer satisfaction. Multi-objective VRP produces a *Pareto frontier* of non-dominated solutions, allowing the decision-maker to choose based on priorities.

## 1.5 Relationship to TSP and Bin Packing I

The VRP is closely related to two other fundamental combinatorial optimisation problems: the **Travelling Salesman Problem (TSP)** and the **Bin Packing Problem (BPP)**.

**TSP is a special case of VRP: one vehicle, no capacity constraint**



## 1.5 Relationship to TSP and Bin Packing II

**VRP reduces to TSP when:** there is only one vehicle ( $K = 1$ ) and no capacity constraint ( $Q = \infty$ , infinity, meaning capacity is unlimited). In this case, the single vehicle must visit all customers in the cheapest order — this is exactly the TSP. Since TSP is NP-hard, the VRP is also NP-hard.



## 1.5 Relationship to TSP and Bin Packing III

**TSP as a lower bound component:** Because any VRP solution is a set of TSP tours (one per route), and because the optimal TSP tour over all customers is a lower bound on VRP cost (ignoring capacity), TSP bounds are widely used in VRP branch-and-bound algorithms.

**Relationship to Bin Packing:** The *customer-to-vehicle assignment* subproblem in the VRP is analogous to bin packing:

- Each customer's demand  $d_i$  is an *item* with size  $d_i$ .
- Each vehicle of capacity  $Q$  is a *bin* of size  $Q$ .
- The problem: pack all demands into the minimum number of bins (vehicles) without exceeding bin capacity.

## 1.5 Relationship to TSP and Bin Packing IV

This *bin packing lower bound* gives the minimum number of vehicles needed:

$$K^* \geq \left\lceil \frac{\sum_{i=1}^n d_i}{Q} \right\rceil$$

where  $\lceil \cdot \rceil$  is the ceiling function (rounds up to the nearest integer). For example, with total demand = 47 and  $Q = 10$ , we need at least  $\lceil 47/10 \rceil = 5$  vehicles.

### Why These Relationships Matter

Understanding the connection to TSP and bin packing allows us to borrow algorithmic ideas from both fields. TSP algorithms provide good initial tours; bin-packing heuristics provide good initial vehicle assignments. Many VRP heuristics alternate between these two subproblems.

## 1.6 Benchmark Instances I

Benchmarking is critical in VRP research: it allows fair comparison of algorithms across research groups. Two sets of benchmark instances have been most widely used.

### Solomon's Benchmark Instances (1987)

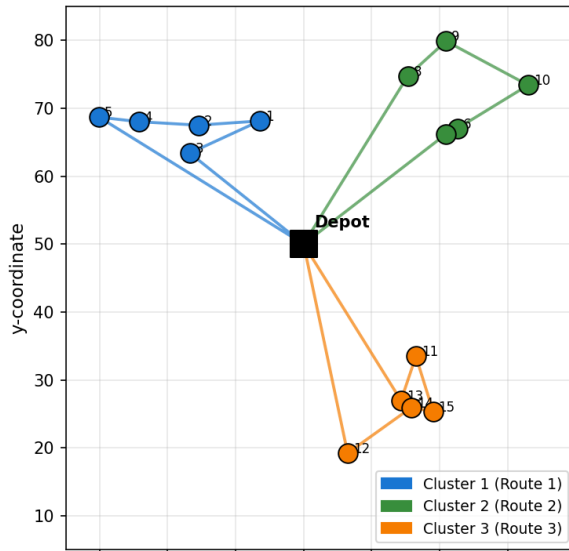
Solomon proposed 56 benchmark instances for the VRPTW, each with  $n = 100$  customers. They are grouped into six types by the spatial distribution of customers and the tightness of time windows:

- **C-type (Clustered):** Customers form distinct geographic clusters. Routes naturally follow cluster boundaries.
- **R-type (Random):** Customers are uniformly distributed at random. Routes have no obvious geographic structure.
- **RC-type (Random-Clustered):** A mix of clustered and random customers.

Within each type, instances **1** have tight time windows and small vehicle capacity (forcing many short routes), while instances **2** have wide time windows and large capacity (permitting few long routes).

## 1.6 Benchmark Instances II

**Solomon C-type Benchmark Instance (Clustered Customers)**  
3 natural clusters => 3 vehicle routes



# Python: Computing CVRP Lower Bounds and Distances

The following Python code implements basic CVRP utility functions: Euclidean distance, bin-packing lower bound on number of vehicles, and the capacity-cut lower bound check.

Listing 1: CVRP utility functions (Python 3)

```
import numpy as np
import math

# -----
# Distance matrix from node coordinates
# -----
def euclidean_distance_matrix(coords):
    """
    coords: list of (x, y) tuples, coords[0] = depot
    Returns: n x n numpy array of Euclidean distances
    """
    n = len(coords)
    D = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            dx = coords[i][0] - coords[j][0]
            dy = coords[i][1] - coords[j][1]
            D[i][j] = math.sqrt(dx*dx + dy*dy)
    return D

# -----
# Bin-packing lower bound on number of vehicles
```

# Python: Nearest Neighbour Heuristic for CVRP

The **Nearest Neighbour (NN) heuristic** is the simplest construction heuristic for CVRP. Starting from the depot, it repeatedly adds the nearest unvisited customer that does not violate the capacity constraint. When no more customers can be added, the vehicle returns to the depot and a new route begins. The NN heuristic is fast ( $O(n^2)$  time) but typically gives solutions 20–30% above optimal. It is often used as a starting point for more sophisticated local search.

Listing 2: Nearest Neighbour heuristic for CVRP

```
def nearest_neighbour_cvrp(coords, demands, capacity):  
    """  
    coords : list of (x,y), coords[0] = depot  
    demands: list of demands, demands[0] = 0 (depot)  
    capacity: vehicle capacity Q  
    Returns : list of routes (each route is a list of node indices)  
    """  
  
    D = euclidean_distance_matrix(coords)  
    n = len(coords)  
    unvisited = set(range(1, n))    # customer indices  
    routes = []  
  
    while unvisited:  
        route = [0]                # start at depot  
        load = 0.0  
  
        while True:  
            current = route[-1]
```

# Python: Clarke-Wright Savings Algorithm

The **Clarke-Wright Savings Algorithm** (Clarke and Wright, 1964) is a classic construction heuristic that builds routes by merging two initially separate “spoke” routes into a combined route whenever doing so yields a positive *saving*.

**Intuition.** If every customer were served by a separate vehicle (spoke routes: depot  $\rightarrow i \rightarrow$  depot), the total cost would be  $\sum_i 2c_{0i}$  (twice the depot-to-customer distance for each customer, where  $c_{0i}$  is the cost from depot to customer  $i$ ). Merging customers  $i$  and  $j$  into one route (depot  $\rightarrow i \rightarrow j \rightarrow$  depot) saves:

$$s(i, j) = c_{0i} + c_{0j} - c_{ij}$$

A positive  $s(i, j)$  means combining  $i$  and  $j$  into one route is cheaper than serving them separately. The algorithm merges routes in decreasing order of saving.

## Listing 3: Clarke-Wright Savings Algorithm

```
def clarke_wright_savings(coords, demands, capacity):
    D = euclidean_distance_matrix(coords)
    n = len(coords)
    depot = 0

    # Compute all savings  $s(i, j) = d(0, i) + d(0, j) - d(i, j)$ 
    savings = []
    for i in range(1, n):
        for j in range(1, n):
            if i < j:
```

# Python: 2-opt Local Search for Route Improvement

The **2-opt** local search algorithm improves a single route by repeatedly removing two edges and reconnecting the route in the only other way that maintains a valid tour. It is the simplest and most widely used VRP improvement heuristic.

Listing 4: 2-opt local search for route improvement

```
def two_opt(route, dist_matrix):
    """
    Improve a single route using 2-opt swaps.
    route: list of nodes including depot at start and end, e.g. [0,2,3,5,0]
    Returns: improved route and its cost
    """
    best = route[:]
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                # Try reversing the segment best[i:j+1]
                new_route = best[:i] + best[i:j+1][::-1] + best[j+1:]
                old_cost = route_cost(best, dist_matrix)
                new_cost = route_cost(new_route, dist_matrix)
                if new_cost < old_cost - 1e-9:
                    best = new_route
                    improved = True
    return best
```



# Chapter 1 Summary: Key Takeaways I

This chapter has introduced the full landscape of vehicle routing problems. The following points summarise the most important concepts.

## 1. The Core Problem: CVRP

The Capacitated VRP is the foundation of the entire field. It asks: given  $n$  customers with demands,  $K$  vehicles of capacity  $Q$ , and a depot, find the minimum-cost set of routes that covers every customer exactly once without violating capacity. The problem is NP-hard (it includes TSP as a special case).

## 2. Mathematical Models

Two key formulations exist:

- *Arc-flow models* (MTZ-style): use binary variables  $x_{ij}$  and auxiliary variables  $u_i$  for subtour elimination.
- *Set-partitioning model*: enumerate all feasible routes; select the minimum-cost covering subset. Solved via column generation.

### 3. The VRP Family

Real-world variants include: VRPTW (time windows), PDP/PDPTW (pickup and delivery), SVRP (stochastic), DVRP (dynamic), VRPP (profits), SDVRP (split deliveries), HVRP (heterogeneous fleet), PVRP (periodic), MDVRP (multi-depot). Each adds one or more constraints or relaxations to the basic CVRP.

## 4. Relationship to TSP and Bin Packing

- VRP with one vehicle and no capacity = TSP.
- Customer-to-vehicle assignment  $\approx$  bin packing.
- Both relationships yield useful lower bounds and algorithmic ideas.

## 5. Benchmarks

- **Christofides-Mingoizzi-Toth (CMT)**: 14 CVRP instances,  $n \in [50, 199]$ ; standard for capacitated routing.
- **Solomon (1987)**: 56 VRPTW instances,  $n = 100$ ; C/R/RC types; standard for time-window routing.

## Chapter 1 Summary: Key Takeaways IV

### Final Takeaway

The Vehicle Routing Problem is one of the most practically important optimisation problems, with billions of dollars of potential savings in global logistics. Its mathematical richness — combining graph theory, integer programming, and combinatorial optimisation — makes it a cornerstone of operations research. Every logistics company, from last-mile delivery startups to global supply chains, faces a VRP variant every single day.

# Selected Bibliography I

The following references are among the most foundational works in the VRP literature, as cited in Chapter 1 of Irnich, Toth, and Vigo (2014).



M.L. Balinski and R.E. Quandt, *On an integer program for a delivery problem*, Operations Research, 12 (1964), pp. 300–304.



G. Clarke and J.W. Wright, *Scheduling of vehicles from a central depot to a number of delivery points*, Operations Research, 12 (1964), pp. 568–581.



N. Christofides, A. Mingozzi, and P. Toth, *The vehicle routing problem*, in Combinatorial Optimisation, Wiley, 1979, pp. 315–338.



M.M. Solomon, *Algorithms for the vehicle routing and scheduling problems with time window constraints*, Operations Research, 35 (1987), pp. 254–265.



P. Toth and D. Vigo (eds.), *The Vehicle Routing Problem*, SIAM Monographs on Discrete Mathematics and Applications, SIAM, Philadelphia, 2002.



S. Irnich, P. Toth, and D. Vigo, *The family of vehicle routing problems*, Chapter 1 in: P. Toth and D. Vigo (eds.), *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed., SIAM, 2014.

# Selected Bibliography II



G. Laporte and Y. Nobert, *Exact algorithms for the vehicle routing problem*, Annals of Discrete Mathematics, 31 (1987), pp. 147–184.



J.-F. Cordeau and G. Laporte, *The dial-a-ride problem: Modeling and solution*, Journal of the Operational Research Society, 58 (2003), pp. 293–307.



M. Gendreau, A. Hertz, and G. Laporte, *New insertion and postoptimization procedures for the traveling salesman problem*, Operations Research, 40 (1992), pp. 1086–1094.

# End of Chapter 1

The Family of Vehicle Routing Problems

*Irnich, Toth, Vigo — Vehicle Routing: Problems, Methods, and Applications, 2nd ed., SIAM 2014*